

Problem Statement

Predict whether the stock price will increase or decrease on the next trading day.

In []:

1 — Import Required Libraries

```
In [1]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

import matplotlib.pyplot as plt
import seaborn as sns
```

2 — Load Dataset

```
In [2]: df = pd.read_csv('feature_engineered_stock_data.csv')
```

```
In [3]: print(df.head())
```

	Date	Close	High	Low	Open	Volume	Company	\
0	2015-01-12	43.0508	43.7375	42.7075	43.7111	2357904	Adani	
1	2015-02-01	71.1080	71.7722	70.7614	70.9491	6565229	Adani	
2	2015-02-02	94.5603	95.6867	92.2930	94.0115	41701833	Adani	
3	2015-02-03	102.5173	103.3982	99.6436	101.3765	8781363	Adani	
4	2015-02-06	91.9898	94.3653	89.7514	92.1342	112566102	Adani	

	Daily_Price_Change	Daily_Return_Percentage	Price_Volatility	\
0	-0.6603	-1.5106	1.0301	
1	0.1589	0.2239	1.0109	
2	0.5488	0.5837	3.3937	
3	1.1408	1.1254	3.7547	
4	-0.1444	-0.1567	4.6139	

	High_Low_Percentage	Rolling_STD_7	Previous_Close	Close_Lag_3	\
0	2.4119	22.9254	42.5490	47.0521	
1	1.4286	22.9524	43.0508	38.0326	
2	3.6770	24.2843	71.1080	42.5490	
3	3.7681	26.8225	94.5603	43.0508	
4	5.1408	27.8137	102.5173	71.1080	

	Day_of_Week	Month	Price_Range_Ratio	Closing_Price_Difference
0	Monday	January	0.0239	0.5018
1	Sunday	February	0.0142	28.0571
2	Monday	February	0.0359	23.4523
3	Tuesday	February	0.0366	7.9570
4	Friday	February	0.0502	-10.5276

3 — Dataset Information

In [4]: `print(df.info())`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2910 entries, 0 to 2909
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Date                                  2910 non-null   object
1   Close                                2910 non-null   float64
2   High                                 2910 non-null   float64
3   Low                                  2910 non-null   float64
4   Open                                 2910 non-null   float64
5   Volume                               2910 non-null   int64
6   Company                              2910 non-null   object
7   Daily_Price_Change                   2910 non-null   float64
8   Daily_Return_Percentage              2910 non-null   float64
9   Price_Volatility                     2910 non-null   float64
10  High_Low_Percentage                  2910 non-null   float64
11  Rolling_STD_7                        2910 non-null   float64
12  Previous_Close                       2910 non-null   float64
13  Close_Lag_3                          2910 non-null   float64
14  Day_of_Week                          2910 non-null   object
15  Month                                2910 non-null   object
16  Price_Range_Ratio                    2910 non-null   float64
17  Closing_Price_Difference              2910 non-null   float64
dtypes: float64(13), int64(1), object(4)
memory usage: 409.3+ KB
None
```

```
In [5]: print(df.shape)
```

```
(2910, 18)
```

```
In [6]: print(df.isnull().sum())
```

```
Date                0
Close               0
High               0
Low                0
Open               0
Volume             0
Company            0
Daily_Price_Change 0
Daily_Return_Percentage 0
Price_Volatility    0
High_Low_Percentage 0
Rolling_STD_7       0
Previous_Close      0
Close_Lag_3         0
Day_of_Week         0
Month              0
Price_Range_Ratio   0
Closing_Price_Difference 0
dtype: int64
```

4 — Convert Date Column

Why Important:- Machine learning and time-series operations require proper date format.

```
In [7]: df['Date'] = pd.to_datetime(df['Date'])
```

5 — Sort Dataset Chronologically

Why Important:-

Stock data depends on historical order.

Incorrect order creates unrealistic predictions.

```
In [8]: df = df.sort_values(by='Date')
```

6 — Create Target Variable

Create prediction output column.

```
In [9]: df['Target'] = (df['Close'].shift(-1) > df['Close']).astype(int)
```

7 — Remove Last Row

Why Important:-

Last row has no future value available.

```
In [10]: df = df[:-1]
```

8 — Encode Categorical Variables

Convert text columns into numerical values.

Why Important:- Machine learning models require numerical input.

```
In [11]: encoder = LabelEncoder()

categorical_columns = ['Company', 'Day_of_Week', 'Month']

for col in categorical_columns:
    df[col] = encoder.fit_transform(df[col])
```

9 — Feature Selection

Separate features and target.

Why Date Is Removed:-

Date itself does not directly help prediction.

```
In [12]: X = df.drop(['Target', 'Date'], axis=1)
y = df['Target']
```

10 — Chronological Train-Test Split

Split dataset without shuffling.

Why Important:-

Prevents future information leakage.

Creates realistic stock prediction environment.

```
In [13]: split_index = int(len(df) * 0.8)
```

```
In [14]: X_train = X[:split_index]
X_test = X[split_index:]
```

```
In [15]: y_train = y[:split_index]
y_test = y[split_index:]
```

11 — Train Logistic Regression

Train baseline model.

```
In [16]: lr_model = LogisticRegression(max_iter=1000)
```

```
In [17]: lr_model.fit(X_train, y_train)
```

C:\Users\patil\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\linear_model_logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

```
Out[17]:
```

▼ LogisticRegression ⓘ ?

► Parameters

12 — Train Decision Tree

Train tree-based model.

```
In [18]: dt_model = DecisionTreeClassifier(random_state=42)
```

```
In [19]: dt_model.fit(X_train, y_train)
```

```
Out[19]:
```

▼ DecisionTreeClassifier ⓘ ?

► Parameters

13 — Train Random Forest

Why Random Forest Is Important:-

Better accuracy, Less overfitting, Handles complex stock behavior, Train advanced ensemble model.

```
In [20]: rf_model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)  
  
rf_model.fit(X_train, y_train)
```

```
Out[20]:
```

▼ RandomForestClassifier ⓘ ?

► Parameters

14 — Make Predictions

Generate predictions.

```
In [21]: lr_pred = lr_model.predict(X_test)
```

```
In [22]: dt_pred = dt_model.predict(X_test)
```

```
In [23]: rf_pred = rf_model.predict(X_test)
```

15 — Evaluate Logistic Regression Model

Measure the performance of the Logistic Regression model.

```
In [24]: print("Logistic Regression Performance")

print("Accuracy:",
      accuracy_score(y_test, lr_pred))
```

Logistic Regression Performance
Accuracy: 0.5378006872852233

```
In [25]: print("Precision:",
              precision_score(y_test, lr_pred))
```

Precision: 0.813953488372093

```
In [26]: print("Recall:",
              recall_score(y_test, lr_pred))
```

Recall: 0.11824324324324324

```
In [27]: print("F1 Score:",
              f1_score(y_test, lr_pred))
```

F1 Score: 0.20648967551622419

16 — Evaluate Decision Tree Model

Measure the performance of the Decision Tree model.

```
In [28]: print("Decision Tree Performance")

print("Accuracy:",
      accuracy_score(y_test, dt_pred))
```

Decision Tree Performance
Accuracy: 0.7199312714776632

```
In [29]: print("Precision:",
              precision_score(y_test, dt_pred))
```

Precision: 0.734982332155477

```
In [30]: print("Recall:",
              recall_score(y_test, dt_pred))
```

Recall: 0.7027027027027027

```
In [31]: print("F1 Score:",  
            f1_score(y_test, dt_pred))
```

F1 Score: 0.7184801381692574

17 — Evaluate Random Forest Model

Measure the performance of the Random Forest model.

```
In [32]: print("Accuracy:",  
            accuracy_score(y_test, rf_pred))
```

Accuracy: 0.7628865979381443

```
In [33]: print("Precision:",  
            precision_score(y_test, rf_pred))
```

Precision: 0.8526785714285714

```
In [34]: print("Recall:",  
            recall_score(y_test, rf_pred))
```

Recall: 0.6452702702702703

```
In [35]: print("F1 Score:",  
            f1_score(y_test, rf_pred))
```

F1 Score: 0.7346153846153847

18 — Feature Importance Analysis

Identify which features are most important for prediction.

```
In [36]: importance = pd.DataFrame({  
        'Feature': X.columns,  
        'Importance': rf_model.feature_importances_  
    })  
  
importance = importance.sort_values(  
    by='Importance',  
    ascending=False  
)  
  
importance['Importance'] = importance['Importance'].round(4)  
  
print(importance)
```

	Feature	Importance
5	Company	0.2188
0	Close	0.1083
1	High	0.1010
2	Low	0.0847
11	Previous_Close	0.0786
3	Open	0.0728
4	Volume	0.0675
10	Rolling_STD_7	0.0540
12	Close_Lag_3	0.0417
8	Price_Volatility	0.0332
16	Closing_Price_Difference	0.0248
6	Daily_Price_Change	0.0233
7	Daily_Return_Percentage	0.0229
9	High_Low_Percentage	0.0215
15	Price_Range_Ratio	0.0213
14	Month	0.0147
13	Day_of_Week	0.0110

19 — Train XGBoost Model

Train advanced boosting model for stock prediction.

Why XGBoost Is Important:-

Powerful boosting algorithm, High-performance ML model, Very popular in financial prediction, Good at handling complex stock market patterns,

```
In [37]: from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=5,
    random_state=42
)

xgb_model.fit(X_train, y_train)

print("XGBoost model trained successfully")
```

XGBoost model trained successfully

20— Generate XGBoost Predictions

Use trained XGBoost model to predict stock movement.

```
In [38]: xgb_pred = xgb_model.predict(X_test)
```

21 — Evaluate XGBoost Performance

Check model performance.

```
In [39]: print("XGBoost Performance")

print("Accuracy:",
      accuracy_score(y_test, xgb_pred))
```

XGBoost Performance
Accuracy: 0.7697594501718213

```
In [40]: print("Precision:",
              precision_score(y_test, xgb_pred))
```

Precision: 0.8068181818181818

```
In [41]: print("Recall:",
              recall_score(y_test, xgb_pred))
```

Recall: 0.7195945945945946

```
In [42]: print("F1 Score:",
              f1_score(y_test, xgb_pred))
```

F1 Score: 0.7607142857142857

22 — Feature Importance Analysis

Identify which features are most important for prediction.

```
In [43]: xgb_importance = pd.DataFrame({

        'Feature': X.columns,

        'Importance': xgb_model.feature_importances_

    })

xgb_importance = xgb_importance.sort_values(
    by='Importance',
    ascending=False
)

xgb_importance['Importance'] = (
    xgb_importance['Importance'].round(4)
)

print(xgb_importance)
```

	Feature	Importance
5	Company	0.7022
10	Rolling_STD_7	0.0781
0	Close	0.0271
3	Open	0.0253
11	Previous_Close	0.0231
12	Close_Lag_3	0.0159
1	High	0.0151
2	Low	0.0140
6	Daily_Price_Change	0.0130
7	Daily_Return_Percentage	0.0128
9	High_Low_Percentage	0.0116
8	Price_Volatility	0.0115
4	Volume	0.0114
15	Price_Range_Ratio	0.0114
14	Month	0.0101
13	Day_of_Week	0.0089
16	Closing_Price_Difference	0.0084

23— Compare All Models

Find best-performing model.

```
In [44]: results = pd.DataFrame({

    'Model': [
        'Logistic Regression',
        'Decision Tree',
        'Random Forest',
        'XGBoost'
    ],

    'Accuracy': [
        accuracy_score(y_test, lr_pred),
        accuracy_score(y_test, dt_pred),
        accuracy_score(y_test, rf_pred),
        accuracy_score(y_test, xgb_pred)
    ],

    'Precision': [
        precision_score(y_test, lr_pred),
        precision_score(y_test, dt_pred),
        precision_score(y_test, rf_pred),
        precision_score(y_test, xgb_pred)
    ],

    'Recall': [
        recall_score(y_test, lr_pred),
        recall_score(y_test, dt_pred),
        recall_score(y_test, rf_pred),
        recall_score(y_test, xgb_pred)
    ],

    'F1 Score': [
        f1_score(y_test, lr_pred),
        f1_score(y_test, dt_pred),
        f1_score(y_test, rf_pred),
        f1_score(y_test, xgb_pred)
    ]
})
```

```

}))

results = results.round(4)

print(results)

```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.5378	0.8140	0.1182	0.2065
1	Decision Tree	0.7199	0.7350	0.7027	0.7185
2	Random Forest	0.7629	0.8527	0.6453	0.7346
3	XGBoost	0.7698	0.8068	0.7196	0.7607

24 — Save All Models

Store trained models permanently.

```

In [45]: import joblib

joblib.dump(
    lr_model,
    'logistic_regression_model.pkl'
)

joblib.dump(
    dt_model,
    'decision_tree_model.pkl'
)

joblib.dump(
    rf_model,
    'random_forest_model.pkl'
)

joblib.dump(
    xgb_model,
    'xgboost_model.pkl'
)

print("All models saved successfully.")

```

All models saved successfully.

```

In [46]: results.to_csv(
    'model_comparison_results.csv',
    index=False
)

print("Comparison table saved.")

```

Comparison table saved.

In []:

In []:

In []: